Natural Language and Information Retrieval
Degree in Data Science
ETSINF. Academic year 2024-2025

Lab 2: Sexism Identification in Twitter

Session 4. Transformers and Explainability

Goal

The goal of this lab session is to provide students with hands-on experience in applying SHAP (SHapley Additive exPlanations) and LIME (Local Interpretable Model-agnostic Explanations) to gain insights into the decision-making process of a fine-tuned transformer-based model. Specifically, the lab focuses on the task of *"EXIST2025: sEXism Identification In Social neTworks".* Particularly in subtask 1, which aims at distinguishing sexist tweets from nonsexist ones, and subtask 2, which aims at determining the authors' intention in sexist messages. Through this session, students will understand the importance of interpretability and explainability in machine learning models, particularly for complex natural language processing tasks. They will learn to apply SHAP and LIME techniques to a transformer-based model, such as BERT, which has been fine-tuned for sexi*sm Identification in Twitter*. They will gain insights into the model's decision-making process by analyzing feature importance and identifying the most relevant input features (e.g., words, phrases, or contextual cues) that contribute to the model's predictions. Moreover, they will practice visualizing and interpreting the explanations generated by SHAP and LIME.

Bibliography

1. Holzinger, A., Saranti, A., Molnar, C., Biecek, P., Samek, W. (2022). Explainable AI Methods - A Brief Overview. In: Holzinger, A., Goebel, R., Fong, R., Moon, T., Müller, KR., Samek, W. (eds) xxAI - Beyond Explainable AI. XAI 2020. Lecture Notes in Computer Science, vol 13200. Springer, Cham. https://doi.org/10.1007/978-3-031-04083-2_2
2. Longo, Luca, et al. Explainable artificial intelligence (XAI) 2.0: A manifesto of open challenges and interdisciplinary research directions. *Information Fusion* (2024): 102301.
3. Ali, Sajid, et al. Explainable Artificial Intelligence (XAI): What we know and what is left to attain Trustworthy Artificial Intelligence. *Information fusion* 99 (2023): 101805.
4. Attanasio, G., Pastor, E., Di Bonaventura, C., & Nozza, D. (2023, May). ferret: a Framework for Benchmarking Explainers on Transformers. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations* (pp. 256-266).
5. Lundberg, Scott M., and Su-In Lee. “A unified approach to interpreting model predictions.” Advances in Neural Information Processing Systems (2017)
6. Marco Ribeiro, Sameer Singh, and Carlos Guestrin. 2016. “Why Should I Trust You?”: Explaining the Predictions of Any Classifier. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Demonstrations*, pages 97–101, San Diego, California. Association for Computational Linguistics.

Introduction to eXplainable AI (XAI)

Machine learning models, especially deep neural networks, have achieved remarkable performance across various tasks. However, these models often operate as "black boxes", making understanding how they get their predictions or decisions challenging. This lack of interpretability raises concerns, particularly in high-stakes domains such as healthcare, where a misdiagnosis can have severe consequences, finance, where automated trading decisions can lead to market volatility, and criminal justice, where an incorrect prediction can impact a person's freedom. Moreover, many industries are subject to regulatory requirements that mandate explainability and accountability for automated decision-making systems.

XAI aims to develop techniques and methodologies that can elucidate the inner workings of complex machine learning models, thereby increasing their transparency, trustworthiness, and accountability. By providing interpretable explanations for model predictions, XAI can help people gain insights into the model's prediction process, identify potential biases or errors, and ensure compliance with relevant regulations in the application domain.

While traditional machine learning models prioritize predictive performance, the pursuit of interpretability often introduces a trade-off. Highly accurate but opaque models may need to be sacrificed

for more interpretable but potentially less accurate alternatives. This trade-off between model performance and interpretability is a key consideration in designing and deploying XAI systems, as different applications may have varying requirements for accuracy and explainability. The risks of using opaque models in high-stakes domains include:

- Inability to understand and correct errors.
- Potential for biases to go unnoticed.
- Lack of accountability for the decisions made by these models.

XAI techniques can be broadly categorized into two approaches: local explanations and global explanations.

Local explanations aim to provide insights into individual predictions or decisions made by the model, typically by identifying the most relevant features or inputs that contributed to a specific output. These explanations help understand the rationale behind individual decisions and can help in debugging or identifying potential biases.

Global explanations, on the other hand, seek to provide a holistic understanding of the model's behavior across its entire input space. These explanations may involve extracting interpretable rules or decision trees that approximate the model's decision boundaries or visualizing the model's internal representations. Global explanations can help people better understand the model's

capabilities, limitations, and decision-making patterns, enabling more informed model selection and deployment. A detailed explanation of the different approaches for addressing explainability in machine learning models can be found in [3].

The choice of technique often depends on the specific requirements of the application, the type of model being explained, and the desired level of interpretability.

SHAP (SHapley Additive exPlanations) [5] and LIME (Local Interpretable Model-agnostic Explanations) [6] are not just theoretical concepts, but practical tools that are popular and widely used in the field of machine learning. They are particularly useful for explaining the predictions of machine learning models, providing insights into feature importance and local interpretability. They are considered **Feature Importance Methods, aiming to identify** the most relevant input features or variables contributing to a model's predictions. Particularly, they are useful for explaining decisions made by models operating on images, text, or other structured data.

SHAP method

The SHAP method is based on the concept of Shapley values from cooperative game theory, which is a mathematical framework for studying situations where multiple players interact to produce a certain outcome. It aims to assign an importance value (SHAP value) to each feature for a particular prediction, representing its

contribution to the model's output. The underlying principles and assumptions of SHAP are:

1. **Additive Feature Attribution**: SHAP assumes that the contribution of each feature to the model's prediction can be represented as an additive component.
2. **Local Accuracy**: SHAP ensures that the sum of the SHAP values for all features, combined with the model's expected value (base value), equals the actual model prediction.
3. **Consistency**: SHAP values are consistent with the model's behavior, meaning that if a feature's contribution increases or decreases, its SHAP value should change accordingly.

One of the key strengths of SHAP is its versatility. It can be applied to various model types, including tree-based models, neural networks, and ensemble methods, making them valuable tools for a wide range of machine learning practitioners.

LIME method

LIME is a technique that focuses on generating local explanations for individual predictions. The underlying principles and assumptions of LIME are:

1. **Local Fidelity**: LIME approximates the model's behavior in the vicinity of the instance being explained by fitting a simple, interpretable model (e.g., linear model or decision tree) to the local neighborhood.

2. **Interpretable Representation**: LIME represents the explanation in an interpretable form, such as feature importances or decision rules, making it easy for humans to understand.
3. **Model Agnostic**: LIME can be applied to any machine learning model, treating it as a black box and only requiring access to the model's predictions.
4. **Perturbation and Proximity**: LIME perturbs the input data around its neighborhood to understand how the model's predictions change. It then weighs these perturbed data points based on their proximity to the original example and learns an interpretable model based on these perturbations.

LIME generates local explanations by perturbing the input data and observing the changes in the model's predictions. It then fits an interpretable surrogate model to the perturbed data, effectively approximating the complex model's behavior locally.

Differences, Strengths, and Weaknesses

While both SHAP and LIME aim to provide feature importance and local explanations, they differ in several aspects:

1. **Scope**: SHAP provides both local and global explanations, while LIME focuses solely on local explanations.
2. **Approach**: SHAP uses a game-theoretic approach based on Shapley values, while LIME relies on fitting interpretable surrogate models.

3. **Assumptions**: SHAP assumes additive feature attribution and consistency, while LIME assumes local fidelity and interpretability.
4. **Computational Complexity**: SHAP can be computationally expensive, especially for complex models and large datasets, while LIME is faster and more scalable.
5. **Model Dependence**: SHAP can leverage model-specific properties for specific model types, potentially providing more accurate explanations, while LIME is truly model-agnostic.

SHAP is widely used in applications where both local and global interpretability is desired, such as in healthcare for understanding the factors influencing a patient's diagnosis. It is particularly useful for tree-based models and certain types of neural networks. On the other hand, LIME is often preferred when model-agnostic explanations are required, such as in finance for explaining the factors affecting a stock's price, or when computational efficiency is a concern. It can be applied to many models, including complex deep-learning architectures.
Both SHAP and LIME have their strengths and weaknesses. The choice between them often depends on the specific requirements of the application, the type of model being explained, and the trade-off between explanation quality and computational efficiency

FERRET-XAI Library

The FERRET-XAI (eXplainable AI Feature Exploration and Robustness Toolkit) allows the SHAP and LIME methods to be used efficiently. It is an open-source Python library that provides a unified interface for many explainability techniques, including SHAP and LIME. The library is deeply integrated with the transformer's library, models, and code. It aims to simplify the process of generating explanations for machine learning models and facilitate the analysis of model behavior and robustness.

- **Model Robustness Analysis**: FERRET-XAI allows users to evaluate the robustness of their models by analyzing the stability of explanations under different perturbations and input conditions.
- **Feature Analysis**: The library provides tools for feature importance analysis, feature interaction analysis, and feature visualization, enabling users to gain deeper insights into the behavior of their models.
- **Extensibility**: The library is designed to be extensible, allowing users to integrate their custom explainability techniques or models.

By providing a unified interface for SHAP, LIME, and other explainability methods, FERRET-XAI simplifies the process of generating explanations and analyzing model behavior. It offers a comprehensive set of tools for interpretability, robustness analysis, and feature exploration, making it a valuable resource for

researchers, developers, and practitioners working in eXplainable AI.

Installing FERRET-XAI

To install the library you can use pip, the package installer for Python. Follow these steps:

1. Prerequisites: Make sure you have Python installed on your system. FERRET-XAI requires Python 3.6 or later.
2. Install FERRET-XAI: You can install the latest stable version of FERRET-XAI from PyPI (Python Package Index) using the following command:

```
!pip install -U ferret-xai
```

In you have trouble installling ferret-xai library, maybe you have to install additional packages. Have a look to the notebook of the session.

To illustrate how to obtain explanations, visualize attribution scores with LIME and SHAP methods, and understand the model's outcomes for correct and incorrect classifications, a BERT model was trained for sexism classification, subtask 1 in the EXIST2025 shared task. The pipeline proposed in the previous lab was followed, using the default hyperparameters and training the model for 10 epochs. After training, predictions were obtained on the development sets for English. Four tweets were selected from these predictions: two in which the model made correct

predictions—one for a "*sexism*" tweet and one for a "*non-sexism*" tweet—and two where the model misclassified the tweets—one tweet annotated as "*sexism*" predicted as "*non-sexism*" and one tweet annotated as *"non-sexism*" predicted as "s*exism*".

EXAMPLE 1: Correct prediction for a "*sexism*" tweet using LIME and SHAP

```
from ferret import Benchmark
from ferret.explainers.shap import SHAPExplainer
from ferret.explainers.lime import LIMEExplainer
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
#Defining the explainer to use
l = LIMEExplainer(model, tokenizer)
s = SHAPExplainer(model, tokenizer)


sexism_ok = "aw man she dat one good girl worth a thousand bitches. we'll see in the long run."
bench = Benchmark(model, tokenizer, explainers=[s,l])
explanations = bench.explain(sexism_ok, target=1)
explanations = deduplicate(explanations)
bench.show_table(explanations)
```

Token	aw	man	she	dat	one	good	girl	worth	a	thousand	bitch	##es	._1	we	'	ll	see	in	the	long	run	._2
Partition SHAP	0.05	0.04	0.08	-0.02	0.02	0.05	0.26	0.00	0.00	-0.00	0.38	0.09	-0.00	-0.00	0.00	0.00	-0.00	-0.00	0.00	-0.00	-0.00	-0.00
LIME	0.01	0.06	-0.01	0.02	0.02	0.03	0.27	-0.04	-0.01	-0.06	0.29	-0.02	0.00	-0.04	-0.01	-0.02	0.00	0.03	-0.03	0.01	0.01	0.03

EXAMPLE 2: Correct prediction for a *"non-sexism"* tweet using LIME and SHAP

```
from ferret import Benchmark
from ferret.explainers.shap import SHAPExplainer
from ferret.explainers.lime import LIMEExplainer
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

#Defining the explainer to use
l = LIMEExplainer(model, tokenizer)
s = SHAPExplainer(model, tokenizer)

nosexism_ok = "7 just completed my last final, i’m officially a free woman lets gooooo"
explanations = bench.explain(nosexism_ok, target=0)
explanations = deduplicate(explanations)
bench.show_table(explanations)
```

Token	7	just	completed	my	last	final	,	i	’	m	officially	a	free	woman	lets	goo	##oo	##o
Partition SHAP	0.00	-0.01	0.00	0.01	0.01	-0.01	-0.01	-0.01	0.01	0.08	0.04	0.04	0.31	-0.33	-0.13	-0.00	0.00	0.00
LIME	0.02	-0.01	0.04	-0.01	-0.01	0.04	-0.02	0.02	-0.01	0.03	0.05	0.02	0.31	-0.18	-0.08	-0.07	-0.03	-0.03

EXAMPLE 3: Wrong prediction for a "*sexism*" tweet using LIME and SHAP

```
from ferret import Benchmark
from ferret.explainers.shap import SHAPExplainer
from ferret.explainers.lime import LIMEExplainer
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
#Defining the explainer to use
l = LIMEExplainer(model, tokenizer)
s = SHAPExplainer(model, tokenizer)

sexism_wrong="@djxmenez yea back when i didn’t even have one to slap, unfortunate !"
explanations = bench.explain(sexism_wrong, target=0)
explanations=deduplicate(explanations)
bench.show_table(explanations)
```

Token	@	dj	##x	##men	##ez	ye	##a	back	when	i	didn	’	t	even	have	one	to	slap	,	unfortunate	!
Partition SHAP	-0.01	0.02	0.01	-0.30	0.29	-0.05	0.05	0.02	-0.02	0.07	-0.04	0.01	-0.05	-0.00	-0.01	-0.00	0.01	0.02	0.00	0.01	-0.00
LIME	-0.01	0.08	0.10	-0.14	0.12	-0.07	-0.01	0.09	0.01	0.07	0.02	-0.00	-0.03	-0.02	-0.00	0.04	-0.02	0.09	0.03	-0.01	0.04

EXAMPLE 4: Wrong prediction for a "*non-sexism*" tweet using LIME and SHAP

```
from ferret import Benchmark
from ferret.explainers.shap import SHAPExplainer
from ferret.explainers.lime import LIMEExplainer
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
#Defining the explainer to use
l = LIMEExplainer(model, tokenizer)
s = SHAPExplainer(model, tokenizer)


nosexism_wrong= "Yo I swear sagittarius men and women are the funniest 😭😭😭😭😂😂😂😂😂😂😭😭😭😭😭"
explanations = bench.explain(nosexism_wrong, target=1)
explanations = deduplicate(explanations)
bench.show_table(explanations)
```

Token	yo	i	swear	sa	##git	##tar	##ius	men	and	women	are	the	fun	##nies	##t	[UNK]
Partition SHAP	-0.02	-0.05	0.02	0.01	0.03	-0.00	-0.02	0.05	-0.02	0.30	0.11	0.07	0.23	-0.05	0.01	-0.03
LIME	-0.01	-0.01	0.07	0.01	0.07	0.03	-0.08	-0.02	-0.04	0.17	0.14	0.01	0.20	-0.08	0.06	-0.01

Parameters for the Benchmark Class

- *Model*: The transformer model to be used for benchmarking.
- *Tokenizer*: The tokenizer associated with the transformer model.
- *Task_name*: The name of the task to be performed. The default is "text-classification".
- *Explainers*: A list of explainers to be used for model interpretation. If not specified (List = None), the class will use all explainers implemented in the library.

Parameters for the explain Method of the Benchmark Class

- *Text*: The text to be analyzed using the explainability methods.
- *Target*: The target class for which the explainability analysis should be performed. Defaults to 1.

ACTIVITY

Use the best transformer models that you have fine-tuned—one for Spanish and other for English—specifically trained to address subtask 1 in the *"EXIST2025: sEXism Identification In Social neTworks"* shared task. You must obtain the model predictions for the development set. Consequently, you must identify **three examples** where the model misclassified *"sexism"* **as** *"non-sexism"*. Moreover, you must identify **three examples** where the model misclassified *"non-sexism"* as *"sexism".*

For each of the **six misclassified examples**, use the SHAP and LIME methods provided by the *FERRET-XAI* library to explain the model's predictions. Visualize the attribution scores obtained from SHAP and LIME for these examples. A detailed analysis must be conducted using SHAP and LIME for each of the six misclassified examples. Visualize the attribution scores obtained from SHAP and LIME for these examples.

Finally, based on attribution scores provided by SHAP and LIME, formulate hypotheses about the reason for the model's misclassifications. Discuss any patterns or common features in the misclassified examples that might have led to the wrong predictions. Present your findings in a well-structured report.

Deliver the source code and draft a comprehensive report explaining your findings.

APPENDIX

Due to a bug in the show_table function for visualizing attribution scores in the FERRET-XAI library, a preprocessing step is required to avoid duplicate tokens. To address this issue, a deduplication function must be applied to rename any duplicate tokens.

```
from collections import Counter # Counter counts the number of occurrences of each item
from itertools import tee, count

def uniquify(seq, suffs = count(1)):
   """Make all the items unique by adding a suffix (1, 2, etc).
   `seq` is mutable sequence of strings.
   `suffs` is an optional alternative suffix iterable.
   """

   not_unique = [k for k,v in Counter(seq).items() if v>1] # so we have: ['name', 'zip']
   # suffix generator dict - e.g., {'name': <my_gen>, 'zip': <my_gen>}
   suff_gens = dict(zip(not_unique, tee(suffs, len(not_unique))))
   for idx,s in enumerate(seq):
       try:
           suffix = str(next(suff_gens[s]))
       except KeyError:
           # s was unique
           continue
       else:
           seq[idx] += suffix


def deduplicate(explanations):
   for i in range(len(explanations)):
       tokens = explanations[i].tokens
       uniquify(tokens, (f'_{x!s}' for x in range(1, 100)))
       explanations[i].tokens=tokens
   return explanations
```